

Algoritmos de escalonamento de processos/threads

1. Introdução

Atualmente é normal executar vários programas ao mesmo tempo em um computador. Imagine que temos um computador com um processador de um único núcleo. Você acha que neste computador poderão ser executados vários programas efetivamente em paralelo, ou seja, ao mesmo tempo? A resposta é não! O sistema operacional precisa realizar a troca de contexto entre um processo e outro o tempo todo, executando cada processo individualmente por uma pequena parcela de tempo. Mas o sistema operacional faz isso tão rápido que temos a falsa impressão de que todos os processos estão sendo executados ao mesmo tempo.

2. Escalonamento de processos

Nos sistemas operacionais, o **escalonador** é o responsável pela escolha do processo que será atendido pelo processador (CPU). Quem fica responsável por alternar a execução de um processo para outro, de forma a garantir que todos continuem a executar do mesmo instante em que foram interrompidos é o **despachante**. Este processo de troca realizado pelo despachante é também chamado de troca de contexto. Todo escalonador utiliza um **algoritmo de escalonamento** e cada algoritmo pode definir políticas de escalonamento diferentes, a fim de atender determinados requisitos.

Um escalonador pode atuar em várias frentes, como:

- **Aumentar a vazão:** tentar processar o maior número de processos possível em uma dada unidade de tempo;
- **Diminuir o tempo de retorno:** o tempo de retorno é o tempo marcado entre o momento em que o processo foi submetido para execução e o momento em que ele foi finalizado.
- **Diminuir o tempo de resposta:** diminuir o tempo entre a emissão de um comando e a obtenção de um resultado.

Todo sistema operacional possui uma **tabela de processos**, que possui uma entrada para cada processo com suas respectivas informações. Essas informações detalhadas do processo são necessárias para realizar a troca de contexto de um processo para outro a fim de garantir a perfeita continuidade do processo que está pronto para ser executado.

Assim, antes de trocar a execução de um processo para outro, o escalonador precisa salvar todas essas informações na tabela de processos. Quando o processo é colocado em execução, as informações são lidas para que a execução possa ser retomada de forma perfeita.

3. Algoritmos de escalonamento

Para desenvolver um algoritmo de escalonamento deve-se ter em mente seus objetivos. Um deles pode ser o **ambiente** em que este algoritmo será executado (sistemas em lote, sistemas interativos ou sistemas de tempo real). **Justiça** é outro objetivo importante: deve-se proporcionar a mesma fatia de tempo para processos de uma mesma categoria, ou seja, não se deve beneficiar um processo em específico. Outro objetivo é **manter todas as partes do sistema ocupadas**: de que adianta escalonar vários processos ao mesmo tempo que exigem muito do processador, se as unidades de entrada e saída ficarão ociosas? O ideal é fazer um balanço escolhendo tanto processos que exigem do processador quanto processos que exigem dos dispositivos de entrada e saída.

Algoritmos de escalonamento podem ser divididos em duas categorias: **algoritmos preemptivos** e **algoritmos não-preemptivos**. O conceito de preempção está relacionado com a característica do escalonador de interromper processos em execução, mesmo se sua execução ainda não estiver terminada. Assim, outro processo poderá ser atendido pelo processador.

Algoritmos não preemptivos permitem a execução de um processo até sua finalização ou até que este gere uma interrupção para dar espaço para outro processo. Algoritmos preemptivos alternam entre a execução de processos por determinadas fatias de tempo.

Considere que cada processo possui as informações relevantes ao escalonador como instante de submissão, tempo de execução e prioridade. Um algoritmo poderá escolher uma ou mais dessas informações para tomar uma

decisão sobre qual processo deverá ser escolhido para ser executado. Existem diversos algoritmos preemptivos e não-preemptivos. Abaixo veremos em resumo alguns dos mais conhecidos.

3.1. Algoritmos não-preemptivos:

Esses algoritmos são usados principalmente em sistemas operacionais em lote.

- **Primeiro a chegar, primeiro a sair – First In, First Out (FIFO):** Neste algoritmo, o escalonador elege o processo a ser executado baseado no instante em que o processo foi submetido para a execução. Como uma fila de banco: o primeiro a chegar é o primeiro a ser atendido. O último a chegar só será atendido depois que todos a sua frente forem atendidos. Assim, o primeiro processo submetido será o primeiro a ser atendido. Os seguintes são eleitos de acordo com o instante de submissão.
- **Tarefa mais curta primeiro – Shortest Job First (SJF):** Este algoritmo escolhe o processo a ser executado baseado no tempo que este processo demorara para executar. Os processos de menor tempo de execução são escolhidos primeiro, a partir da fila de submissão. Ou seja, se na fila de execução está um processo P1 que demora 6 segundos e o processo P2 que demora 3 segundos, o processo P2 será o escolhido pois possui o menor tempo de execução. Mas pense: se vários processos pequenos são submetidos constantemente, o que acontece com os processos maiores já submetidos? Os processos maiores poderão nunca ser atendidos: estarão sujeitos a *starvation*, ou seja, poderão “morrer de fome”.

3.2. Algoritmos preemptivos:

Esses algoritmos são usados principalmente em sistemas operacionais interativos.

- **Chaveamento circular (Round Robin):** É um dos algoritmos mais simples, justos e amplamente usados. A cada processo é atribuído um intervalo de tempo de execução (quantum). Se ao final do quantum o processo ainda estiver executando, a CPU sofrerá preempção e o processo seguinte será executado. O processo que acabou de ser executado irá para o fim da fila. A fila de processos é circular, ou seja, ao terminar o quantum do último processo da fila, inicia-se novamente a execução da fila a partir do primeiro processo. Agora pense que o processador acaba gastando uma pequena fração de tempo com a troca de contexto de processos. O que acontece quando o quantum de cada processo for muito pequeno? E se o quantum for muito grande?
- **Escalaonamento por prioridades:** Neste algoritmo, o escalonador escolhe qual processo executar baseando-se na prioridade dos processos submetidos para execução. Processos com prioridades maiores são atendidos primeiro. Para evitar que apenas os processos prioritários sejam executados, o escalonador poderá reduzir a prioridade de um processo após sua execução, para dar chance de execução aos processos com prioridades mais baixas.
- **Escalaonamento garantido:** O objetivo do escalaonamento garantido é “prometer” que cada processo terá determinada fração de tempo de CPU. Desta forma, no momento em que um processo foi criado houver outros 9 na tabela de processos, haverá um total de 10 processos a serem executados e o novo processo terá 1/10 (10%) da CPU garantido. Pode ser que mais processos sejam submetidos a execução, o que reduzirá o tempo disponível para cada processo. Mas isto poderá ser compensado dando fatias de tempo maior para compensar os momentos em que o processo não teve seu tempo garantido.
- **Escalaonamento por loteria:** Neste algoritmo, o escalonador distribui “bilhetes” para cada um dos processos e faz um sorteio cada vez que precisa selecionar qual processo será executado. Quanto mais bilhetes tiver um processo, mais chance ele terá de ser executado. Sempre é sorteado um dos bilhetes dados aos processos. Não há filas de processos.

4. Eficiência de um algoritmo de escalaonamento

Para verificar se um algoritmo está atendendo determinados requisitos como maior vazão ou menor tempo de espera, alguns cálculos poderão ser feitos tendo em consideração o instante de submissão e o instante de finalização dos processos em execução.

4.1. Cálculo do tempo de execução do processo (*turnaround time*)

Para saber quanto tempo cada processo gastou efetivamente para executar, precisamos de alguns dados do processo, como o instante final de tempo em que teve sua execução finalizada, o instante de tempo em que foi submetido para execução. Assim, podemos calcular o tempo de execução do processo com a seguinte fórmula:

$$\text{Tempo de execução do processo} = \text{Instante final de execução} - \text{Instante de chegada}$$

Veja o exemplo: um processo P1 que é submetido no instante 1, termina sua execução no instante 13 terá um tempo total de execução de 12 unidades de tempo, pois:

$$13 - 1 = 12$$

4.2.Cálculo do tempo médio de execução do processo

O tempo médio de execução está diretamente ligado a capacidade do sistema em processar tarefas dada uma determinada quantidade de tempo (vazão). O tempo médio de execução dos processos é a soma total dos tempos de execução dos processos dividida pela quantidade de processos. Se os processos P1, P2 e P3 demoraram respectivamente 12, 14 e 21 unidades de tempo cada, o tempo médio de execução dos processos será aproximadamente de 15,67 unidades de tempo pois:

$$(12 + 14 + 21)/3 = 15,67$$

4.3.Cálculo do tempo de espera do processo (*waiting time*)

Para saber quanto tempo cada processo esperou para executar, precisamos de alguns dados do processo, como o instante final de tempo em que teve sua execução finalizada, o instante de tempo em que foi submetido para execução e quanto tempo de duração ele consome para executar. Assim, podemos calcular o tempo de espera de cada processo com a seguinte fórmula:

$$\text{Tempo de espera do processo} = \text{Instante final de execução} - \text{Instante de chegada} - \text{Tempo de duração}$$

Veja o exemplo: um processo P1 que é submetido no instante 1, termina sua execução no instante 13 e possui tempo de duração de 5 unidades de tempo terá um tempo total de espera de 7 unidades de tempo, pois:

$$13 - 1 - 5 = 7$$

4.4.Cálculo do tempo médio de espera do processo

O tempo médio de espera para se executar um processo pode ser encontrado somando o tempo total de espera de cada processo e dividindo o resultado pela quantidade total de processos.

$$\text{Tempo médio de espera} = \text{Soma dos tempos de espera dos processos} / \text{Quantidade de processos}$$

Considere que foram executados três processos:

- P1 com tempo total de espera de 7 unidades de tempo
- P2 com tempo total de espera de 11 unidades de tempo
- P3 com tempo total de espera de 6 unidades de tempo

O tempo médio de espera será de 8 unidades de tempo, dado que:

$$(7 + 11 + 6) / 3 = 8$$

5. Conclusão

Diversos fatores são envolvidos no processo de escolha do próximo processo a ser executado. O ambiente, tipo do sistema operacional e tarefas que serão executadas podem influenciar na escolha do algoritmo. O escalonador deverá atender da melhor maneira possível os objetivos do usuário e do sistema operacional e esta pode ser uma tarefa complexa. Para um mesmo cenário, diferentes valores para o tempo total de execução e tempo de espera médio podem ser obtidos se variarmos os algoritmos escalonadores. Escolher o melhor escalonador possível é uma tarefa complexa que exige conhecimento do cenário e dos objetivos a serem alcançados.

Exercícios:

1. O que difere os algoritmos de escalonamento preemptivos para os não-preemptivos? Explique e dê exemplos de algoritmos dos dois tipos.
2. O escalonador pode trabalhar para atender alguns objetivos de acordo com o cenário em que está inserido. Sobre aumentar a vazão, diminuir o tempo de retorno e diminuir o tempo de resposta em processos, explique sobre o que se tratam essas métricas.
3. Considere um sistema onde são submetidos para execução vários processos que possuem alta prioridade e também curto tempo de duração. O que acontecerá com os processos de maior tempo de duração que estão na fila caso o algoritmo de escalonamento utilizado for o *Shortest Job First* (SJF)? O que acontecerá com os processos de menor prioridade caso seja utilizado um escalonamento por prioridades?
4. A Tabela 1 abaixo representa um conjunto de processos prontos para utilizar o processador. Para os algoritmos FIFO, SJF (ambos não-preemptivos) e Round Robin (preemptivo), encontre a sequência de execução das tarefas, o tempo médio de processamento e o tempo médio de espera. Insira a ordem em que as tarefas são executadas. Considere que:
 - As trocas de contexto têm duração nula;
 - Para as tarefas de mesma prioridade, use FIFO como critério de desempate;
 - Para tarefas ingressadas em um mesmo instante, a tarefa com menor índice prevalece;
 - Todas as tarefas são orientadas a processamento (*CPU-Bound*);
 - Valores maiores de prioridade indicam menor prioridade.

Tabela 1: Conjunto de tarefas a serem executadas por um processador.

Índice da tarefa	T1	T2	T3	T4	T5
Instante de Ingresso	0	0	3	5	6
Duração	6	5	6	7	5
Prioridade	3	4	6	10	7

Referências:

MACHADO, F. B.; MAIA, L. P. **Arquitetura de sistemas operacionais**. 4.ed. Rio de Janeiro: LTC, 2007.

MARQUES, J. A.; et al. **Sistemas Operacionais. Estudos de caso em Unix, Linux e Windows**. Rio de Janeiro: LTC, 2011.

OLIVEIRA, R. S.; CARISSIMI, A.; TOSCANI, S. **Sistemas operacionais**. 4. ed. Porto Alegre: Bookman, 2010.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 8 Ed. Rio de Janeiro: LTC, 2010.

TANENBAUM, A. S. **Sistemas Operacionais Modernos**. 3. ed. Rio de Janeiro: Prentice Hall Brasil, 2010.